Document Number: P0722R1
Date: 2017-10-12
Reply to: Andrew Hunter ahh@google.com, Richard Smith richardsmith@google.com
Audience: Evolution

Motivation

Efficient sized delete for variable sized classes

Consider the following class:

```
class inlined_fixed_string {
  public:
   inlined_fixed_string() = delete;
   const size_t size() const { return size_; }

   const char *data() const {
     return static_cast<const char *>(this + 1);
   }

   // operator[], etc, with obvious implementations

   inlined_fixed_string *Make(const std::string &data) {
     size_t full_size = sizeof(inlined_fixed_string) + data.size();
     return new(::operator new(full_size))
                  inlined_fixed_string(data.size(), data.c_str());
   }

  private:
   inlined_fixed_string(size_t size, const char *data) : size_(size) {
     memcpy(data(), data, size);
   }
   size_t size_;
};
```

This defines what is (effectively) a variable-sized object, using that to implement an array of size determined at runtime while saving a pointer indirection. (Note: this pattern is simpler (and generally written) with flexible array members, despite their being nonstandard.)

However, what happens when we delete such a string `s` in the presence of sized-delete? Until core issue 2248, the compiler was required to emit a call (equivalent to):

```
::operator delete(s, full_size);
```

which (in reasonable implementations) it has no easy way to do! With that defect resolution, it instead emits:

```
::operator delete(s, sizeof(inlined_fixed_string));
```

which, while a much saner implementation, is in fact incorrect (and will lead to critical failures on any memory allocator which makes use of sized delete information.) The correct fix is to declare an implementation of `operator delete` for the class:

```
static void operator delete(void* ptr) {
  ::operator delete(ptr);
}
```

This allows the class to be safely used in the presence of sized-delete... but gives up any performance advantages of sized delete! Ideally, what we'd like to do is rely on our knowledge of the class's implementation:

```
static void operator delete(void* ptr) {
  inlined_fixed_string *s = reinterpret_cast<inlined_fixed_string *>(ptr);
  size_t full_size = sizeof(inlined_fixed_string) + s->size();
  ::operator delete(ptr, full_size);
}
```

But this is undefined behavior--the destructor has run by the time this operator is invoked, and we can no longer touch any class members. (This despite knowing, since we wrote both, that the destructor did not damage the `size_` member.)

This paper proposes a modification to `operator delete` making the (moral) equivalent of the above code safe.

Delete for objects with extra preceding storage

Consider this class, used to model a use-list for values in a compiler:

```
class User {
  unsigned NumUses;
public:
  User(unsigned NumUses) : NumUses(NumUses) { /*construct Uses*/ }
  virtual ~User();

  void *operator new(size_t Size, unsigned NumUses) {
    size_t ExtraStorage = NumUses * sizeof(Use);
    return static_cast<char*>(::operator new(ExtraStorage + Size))
               + ExtraStorage;
  }

  void operator delete(void *p, size_t size) {
    User *user = static_cast<User*>(p);
    unsigned uses = user->NumUses; // undefined behavior
    ::operator delete(static_cast<char*>(user) - uses * sizeof(Use),
                      size + uses * sizeof(Use));
  }

  Use *use_begin() { return reinterpret_cast<Use*>(this) - NumUses; }
  Use *use_end() { return reinterpret_cast<Use*>(this); }
};
```

Here, a list of `Use` objects are allocated prior to each `User` object, in order to reduce the number of allocations performed to allocate a `User`. The extra `Use` objects must precede the `User` object, not follow it, because `User` is the base class in a class hierarchy.

The problem is that `operator delete` must subtract off the size of the front storage before deallocation, but determining the size of that storage would require loading a member of the class, whose lifetime has already ended, resulting in undefined behavior.

Dynamic dispatch without vptrs

Systems with large numbers of objects cannot always afford a vptr for each object, and can spare only a few bits per object to describe the most-derived type.

For such systems, a hand-rolled dynamic dispatch solution may be used instead of virtual functions:

```
class Widget {
  unsigned Kind : 4;
  unsigned OtherThings : 28;
  // ...

public:
  void foo() {
    // (generated by metaprogramming)
    switch (Kind) {
    case WidgetKind::Grommet:
      return static_cast<Grommet*>(this)->foo();
    case WidgetKind::Wingnut:
      return static_cast<Wingnut*>(this)->foo();
    // ...
    }
  }
};

class Grommet : Widget {
  // ...
  void foo();
};
```

This technique works well, with lower storage costs and lower per-call overhead than traditional virtual dispatch, and is used in several production systems, but cannot be applied to the destructor, so the delete operator cannot be safely applied to a `Widget` pointer (because the wrong destructor would be invoked).

Proposed solution

A delete operator must currently have a function type of the form:

```
void operator delete(void *p, <optionally more params>);
```

We propose supporting a second form for a class-specific operator delete in class `X`, which we call a *destroying operator delete*:

```
void operator delete(X *p, std::destroying_delete_t, <optionally more params>);
```

... where `std::destroying_delete_t` is a standard library tag type whose sole purpose is to identify such overloads:

```
namespace std {
  struct destroying_delete_t { /*see below*/ };
  inline constexpr destroying_delete_t destroying_delete(unspecified);
}
```

(Like `std::nullopt_t`, `destroying_delete_t` has neither a default constructor nor an initializer list constructor.)

The only difference between this form and the preceding form is that the destructor for `*p` is not run before `operator delete(p, std::destroying_delete)` is invoked; invocation of the destructor becomes the responsibility of the `operator delete` function.

How this helps

In the `inlined_fixed_string` example, we can add a sized delete operator that does not encounter undefined behavior, by requesting the size prior to destroying the object:

```
void inlined_fixed_string::operator delete(inlined_fixed_string *s,
                                           std::destroying_delete_t) {
  size_t full_size = sizeof(inlined_fixed_string) + s->size();
  s->~inlined_fixed_string();
  ::operator delete(ptr, full_size);
}
```

We can use a similar technique in the `User` example:

```
void User::operator delete(User *user, std::destroying_delete_t,
                           std::size_t derived_size) {
  unsigned uses = user->NumUses; // undefined behavior
  user->~User();
  ::operator delete(static_cast<char*>(user) - uses * sizeof(Use),
                    uses * sizeof(Use) + derived_size);
}
```

Note that we support the size and alignment of the class being implicitly passed into this form of `operator delete`. A destroying operator delete that takes a size or alignment parameter has the same semantics as a regular operator delete for these parameters: the passed size and alignment are those of the most-derived class (only) if the class has a virtual destructor; if the destructor is not virtual and the destroying operator delete takes a size or alignment parameter, the static type of the deleted object is assumed to be equal to its dynamic type, and the size and alignment of that type are provided.

In the `Widget` example, we can use the `Kind` field to pick the correct destructor to run:

```
void Widget::operator delete(Widget *widget, std::destroying_delete_t) {
  switch (widget->Kind) {
  case WidgetKind::Grommet:
    static_cast<Grommet*>(widget)->~Grommet();
    ::operator delete(widget, sizeof(Grommet);
    return;
  case WidgetKind::Wingnut:
    static_cast<Wingnut*>(widget)->~Wingnut();
    ::operator delete(widget, sizeof(Wingnut);
    return;
  // ...
  }
}
```

Implementation details

An implementation of this proposal is available for both the Itanium C++ ABI and the Microsoft C++ ABI in Clang trunk. (The standard library portion is not included; the type `std::destroying_delete_t` must be defined by the user before a destroying operator delete can be declared.)

When the destructor of the class is non-virtual, there is no particular implementation complexity: the destructor call is simply not invoked prior to calling `operator delete`. However, when the class has a virtual destructor, the `operator delete` is selected dynamically, as if it were looked up in the context of the dynamic type.

There are two main C++ ABIs in common use (the "Itanium ABI" and the "Microsoft ABI"), and they make different choices in how this is implemented. In both cases, this proposal can be implemented with no change in the ABI for any existing base class.

Itanium ABI

When a class has a virtual destructor, it gains an additional vtable slot for a "deleting destructor", which performs the complete operation of a `delete p;` expression: invoking the destructor and then invoking the appropriate

`operator delete` function for the most-derived type.

Under this proposal, the member `operator delete` function itself (or a thunk to call it with the appropriate calling convention, if necessary) would simply be placed into the "deleting destructor" vtable slot. The use of an empty, trivially-copyable struct type for `destroying_delete_t` is recommended, as it allows ABIs in common usage to avoid the need for a thunk.

Microsoft ABI

When a class has a virtual destructor, it gains an implicit boolean parameter indicating whether the object should be implicitly deleted after the usual destruction code runs (by invoking the suitable `operator delete` function).

Under this proposal, if the selected `operator delete` function is a destroying operator delete, the branch on the implicit boolean parameter would instead be performed at the start of the destructor, and in the deleting case, the destructor would jump to the `operator delete` function.

Possible future extensions

Array deletion would benefit from an approach similar to that described in this paper. Specifically, array delete of a class type with a non-trivial destructor requires that the implementation can infer the array length (in order to run the right number of destructors). This is typically implemented by injecting an "array cookie" specifying the array length at the start of the allocated storage.

Due to the requirement that the implementation implicitly run the destructors for the array elements, an overloaded class-specific allocator has no opportunity to control the storage and representation of the array length information, even when it could do so from the contents of the array or from extrinsic information (for instance, querying the allocator itself).

We believe that a destroying form of array delete could enable users to take control of this aspect of array destruction (and others), but further design work is required. In particular, there would need to be a mechanism to enable or disable the introduction of an implicit "array cookie" when invoking an array new. We propose to support a destroying operator delete only for the non-array case for now, in order to leave maximal design freedom for a future destroying array delete extension.

Alternatives considered

Use a mechanism other than `delete p;` to delete objects

The obvious and natural alternative to this proposal is to destroy and delete objects with custom deletion semantics by using special-case logic, instead of using `operator delete`. This has a number of disadvantages:

- It violates the principle that user-defined types are used like built-in types: as soon as you use more advanced allocation techniques, you don't get to use delete expressions any more.
- Existing class hierarchies with virtual destructors cannot be transparently extended with derived classes that allocate dynamic leading or trailing storage.
- The local choice of deallocation strategy leaks out to clients of the code. For example, a custom deleter must be specified when using unique*ptr, and make*unique and make_shared can't be used any more.

Note that the standard requires specializations of `default_delete<T>` to have the same effect as calling `delete p;`, so specializing `default_delete` is not a correct alternative in C++17. We could lift that restriction, but that would not help for other (perhaps user-defined) resource management types that use `new` and `delete` to manage objects.

Use a different syntax for a destroying operator delete

We have considered a number of different syntaxes, and consider the proposed syntax to be the simplest, cleanest, and most readable. Other considered options include:

`void operator delete(T *)`

Here, the use of `T *` rather than `void *` would indicate that this is a destroying operator delete. Feedback from EWG review of a prior revision of this proposal was that this is too subtle; the authors of this proposal agree.

`void operator ~delete(T *)`

This syntax is still relatively subtle, and adds a new readability problem, as there is no `~delete` operator in C++. Further, the purpose of a destroying operator delete is to (fully) override the behavior of the `delete` operator, so using a different operator name seems counterintuitive.

`~T delete()`

Spelling a destroying operator delete as a kind of destructor is tempting, but doing so risks creating a false intuition that destructors for subobjects will be implicitly run. This alternative also suffers from the criticism directed at the `operator ~delete` option.

Acknowledgements

Thanks to David Blaikie for the suggestion of using a tag type to identify that an `operator delete` overload is a destroying operator delete.